

enabling authority to be revoked without cascading.

Chronology and Revokes

The timing of a GRANT or REVOKE statement may have a bearing on its impact. For example, in some DBMSs it is possible to grant a privilege to all users except a specific user by issuing the following statements:

```
GRANT DELETE on titles to public;  
COMMIT;  
REVOKE DELETE on titles from userx;
```

The timing of a GRANT or REVOKE statement may have a bearing on its impact.

The first statement grants everyone the authority to delete data from the titles table. Because the REVOKE statement is issued after the GRANT to public, the individual userx is barred from deleting data from this table. Some DBMSs (e.g., DB2) do not permit such exclusions, because the PUBLIC authority will override any revokes; other DBMSs (e.g., Microsoft SQL Server) do permit such exclusions.

The DBA needs to understand exactly how GRANTs and REVOKEs work for each DBMS being administered in order to properly manage privileges on database objects and resources.

Label-Based Access Control

As database systems and applications become more sophisticated, the need for low-level access control to the data becomes more critical. A growing number of DBMSs offer label-based access control (LBAC) to ensure that each piece of data is secured such that only authorized users can perform authorized functions. With LBAC it is possible to support applications that need a more granular security scheme. LBAC can be set up to specify who can read and modify data in individual rows and/or columns.

LBAC provides for a more granular security scheme, specifying who can read and modify data in individual rows and/or columns.

For example, you might want to set up an authorization scenario such that employees can see their own data but no one else's. Or, your authorization

needs might dictate that each employee's immediate manager is able to see his payroll information as well as all of his employees' data, and so on up through the org chart. Setting up such a security scheme is virtually impossible without LBAC.

An administrator must configure the LBAC system by creating security label components, which are database objects used to represent the conditions determining whether a user can access a piece of data. A security policy, composed of one or more security label components, is used to describe the criteria for determining who has access to what data. The security administrator defines the policy by creating security labels that are composed of security label components.

Once created, a security label can be associated with individual columns and rows in a table to protect the data held there. An administrator allows users access to protected data by granting them security labels. When a user tries to access protected data, that user's security label is compared to the security label protecting the data. The protecting label will block some security labels but not others.

Any individual user can hold security labels for multiple security policies. For any given security policy, however, a user can hold at most one label for read access and one label for write access. A security administrator can also grant exemptions to users. An exemption allows users to access protected data that their security labels might otherwise prevent them from accessing. Together, security labels and exemptions are called LBAC credentials.

Any attempted access to a protected column when the LBAC credentials do not permit that access will fail. If users try to read protected rows not allowed by their LBAC credentials, the DBMS simply acts as if those rows do not exist. Even aggregate functions, such as COUNT(*), will ignore rows when the LBAC credentials do not allow read access. This is important because sometimes even having knowledge that the data exists (without being able to access it) must be protected.

To use LBAC, you will need to add a specially named column to act as the security label. The security label column is matched with the multilevel security hierarchy set up by the administrator. For example, you might want to set up a hierarchy using colors, as shown in [Figure 14.3](#). Of course, the hierarchy need not be so complex—you might choose to use something simpler, such as TOP SECRET, SECRET, and UNCLASSIFIED.

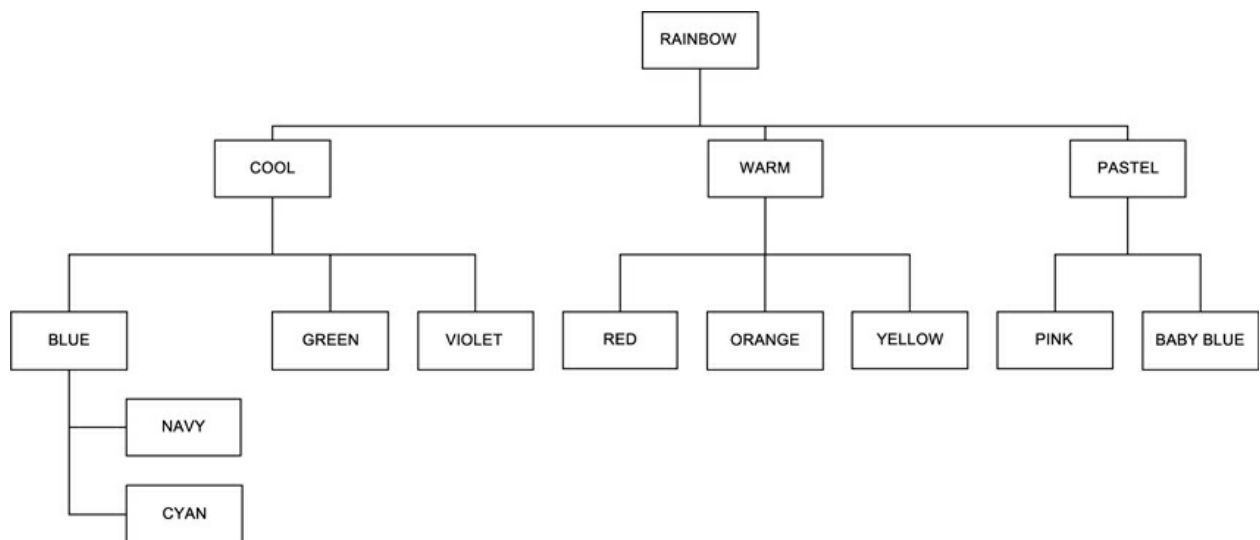


Figure 14.3. Sample LBAC security hierarchy

Consult your DBMS documentation for where and how to establish this hierarchy. For example, with DB2 for z/OS the hierarchy is established in security software, such as RACF. At the top of the hierarchy in the example ([Figure 14.3](#)), the RAINBOW security label includes everything—all colors are included in the rainbow. Middle levels of the hierarchy that represent additional security groupings can be created. In this case we have a second level of COOL, WARM, and PASTEL. Under COOL, we see the cool colors, BLUE, GREEN, and VIOLET. Even further categorization is possible as shown with the various shades of BLUE. And so on throughout the hierarchy. Such a hierarchy provides great flexibility for assigning various levels of security to data in your tables—and thereby appropriately securing the data from unauthorized user access.

Assigning a security label column in your table, and populating that column with the appropriate specification from the hierarchy, makes it possible to control whether or not an individual user can access that particular row. So assigning a row to the security label of RAINBOW makes the data accessible to anyone, whereas assigning the row to NAVY would be much more particular and prohibitive. Users with authority to access RAINBOW can access data because all colors fall under the rainbow. BLUE data can access any row associated with BLUE, or any subordinate blue shade (NAVY and CYAN).

Assigning a security label makes it possible to control whether

or not an individual user can access a particular row.

LBAC offers a very powerful security setup. Such strict security controls are common requirements for government databases, for example.

Security Reporting

Once granted, the DBA will need to monitor and report on the privileges held by users. Database security is maintained in the system catalog. The DBA can use SQL to retrieve the needed information from the appropriate system catalog tables. Alternatively, some DBMSs provide views and system-stored procedures that simplify the retrieval of database security.

As a further consideration, though, be sure to adequately protect the security of the system catalog, especially within the production system. Only the DBA, SA, and security administrator require access to the database security information stored in the system catalog.

Be sure to adequately protect the security of the system catalog.

User security requirements and expectations tend to evolve over time. As new applications are added and business requirements change, database security will need to change. Security reviews should be performed on a regular basis to ensure that database security as implemented continues to match the current user requirements. Reports from the system catalog tables can be used to provide the input for such reviews.

Authorization Roles and Groups

In addition to granting privileges to individual users, the DBMS may provide the capability to assign users

- Specific privileges to a role, which is then granted to others, or
- Specific built-in groups of privileges

Of course, the terminology is not strict among the major DBMSs. Some DBMSs refer to roles as groups, and vice versa. As a DBA, you will need to understand how each DBMS you manage implements roles and groups and how each of these features can be used to simplify database security administration.